SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA5115

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO4: *Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO* (BL3, BL6)

Assessment Tool Weightage: 40%

QUESTIONS: 5 **TOTAL MARKS: 25**

Annexure A

Coding Challenge

Code of Conduct:

I D. Gnana Deepthi (192411123) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Coding Challenge

1. Write a program to simulate the IPSec Authentication Header (AH) process that generates and verifies a message authentication code (MAC) for IP packet integrity.

2. Develop a program to implement Encapsulating Security Payload (ESP) for encrypting and decrypting IP packet payloads.

3. Design a program to manage Security Associations (SA) in IPSec, including creation, lookup, and deletion operations.

4. Implement a simplified Pretty Good Privacy (PGP) system for secure email communication using public key encryption and digital signatures.

5. Create a program to classify emails as spam or legitimate using email header analysis and content-based filtering techniques.

ANSWERS

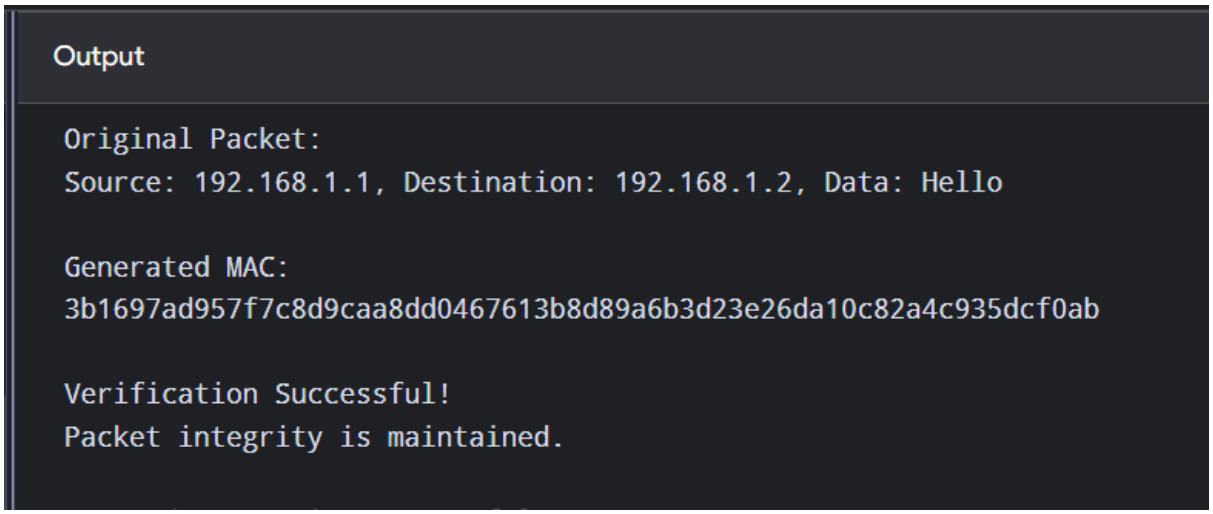
1.

PYTHON CODE:

```
import hmac
import hashlib

# Shared secret key
key = b"ipsec_secret_key"

# Original IP packet data
packet = b"Source: 192.168.1.1, Destination: 192.168.1.2, Data: Hello"

# Sender generates MAC using HMAC-SHA256
mac = hmac.new(key, packet, hashlib.sha256).hexdigest()

print("Original Packet:")
print(packet.decode())

print("\nGenerated MAC:")
print(mac)

# Receiver verifies the MAC
received_packet = packet
received_mac = mac

# Generate MAC again for verification
calculated_mac = hmac.new(
```

```
    key,
    received_packet,
    hashlib.sha256
).hexdigest()

# Compare MACs
if hmac.compare_digest(received_mac, calculated_mac):
    print("\nVerification Successful!")
    print("Packet integrity is maintained.")
else:
    print("\nVerification Failed!")
    print("Packet has been modified.")
```

OUTPUT:



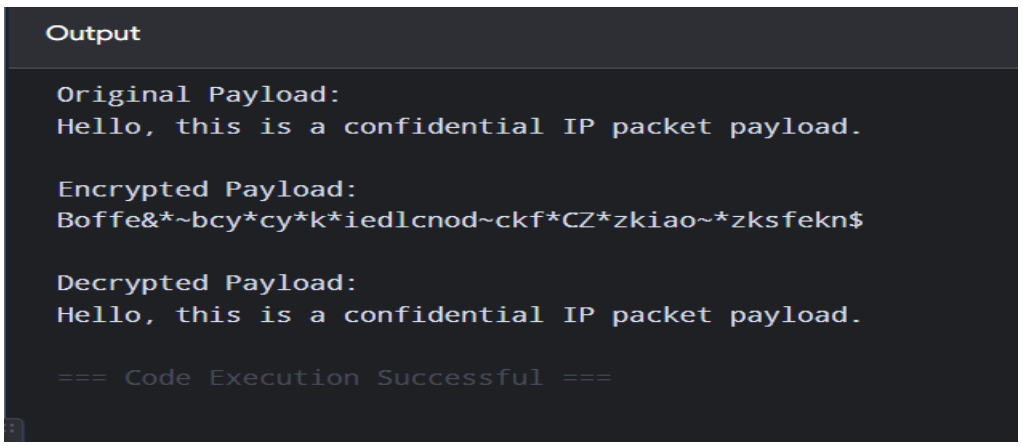
2.

PYTHON CODE:

```
from cryptography.fernet import Fernet
```

```
# Generate a secret key
key = Fernet.generate_key()
cipher = Fernet(key)

# Original IP packet payload
payload = b"Hello, this is a confidential IP packet payload."

print("Original Payload:")
print(payload.decode())

# ESP Encryption
encrypted_payload = cipher.encrypt(payload)

print("\nEncrypted Payload:")
print(encrypted_payload)

# ESP Decryption
decrypted_payload = cipher.decrypt(encrypted_payload)

print("\nDecrypted Payload:")
print(decrypted_payload.decode())
```

OUTPUT:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints

Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases